Can a Machine Play Connections?

Embedding Approaches to the NYT Connections Puzzle

Applied Machine Learning | CU Boulder

1. Introduction

This project investigates whether the use of sentence transformers and greedy algorithms together can solve the New York Times Connections puzzle, a daily word game where players must sort 16 words into 4 hidden groups of 4. The idea is that while computers can not reason or understand language that humans speak, they can convert it into numeric values that we can use to have it draw semantic conclusions.

2. Dataset

2.1 Data Source

The dataset for this project sourced from a public GitHub repository maintained by the user 'Eyefyre', which finds the daily NYT Connections puzzles and is updated automatically via a GitHub Actions workflow running daily. The dataset was fetched directly in Google Colab:

```
!wget
https://raw.githubusercontent.com/Eyefyre/NYT-Connections-Answers/main/con
nections.json
```

2.2 Structure

Each puzzle contains exactly 16 words divided into 4 groups of 4. Each group carries a difficulty level (0 = easiest, 3 = hardest) and a category name. Some entries (clarified in the github ReadMe) have -1 as a difficulty because the data on puzzle difficulty was missing. Here is roughly the dataset we have extracted:

```
CategoryData = [p for p in data if all(a['level'] != -1 for a in
p['answers'])]
print(f"Total puzzle count: {len(data)} | Category puzzle count:
{len(CategoryData)}")
all_words = list(set(
   word for puzzle in data
   for ans in puzzle['answers']
   for word in ans['members']
```

```
))
word2idx = {w: i for i, w in enumerate(all_words)}
print(f"Vocab size: {len(all_words)}")
```

- Total puzzles: 1044
- Words per puzzle: 16
- Groups per puzzle: 4
- Unique words in dataset: 7574

3. Approach

3.1 Semantic Embedding

In order to have a machine solve this puzzle game, we need to find a way to map each word to a numeric value. Each word in a puzzle is mapped to a dense vector representation using a pretrained sentence embedding model I found for free on HuggingFace. The model itself is labeled as a sentence transformer but it will work fine for the purpose of comparing single words to each other.

This model was made as the embedding backbone:

- all-mpnet-base-v2 (768-dim): A strong general-purpose sentence transformer.

Here's an example of us loading up the model here:

```
model = SentenceTransformer('sentence-transformers/all-mpnet-base-v2')
embedding_matrix = model.encode(all_words, show_progress_bar=True)
print(f"Embedding matrix: {embedding_matrix.shape}")
Embedding matrix: (7574, 768)
```

As you can see we have put all 7000 of our words into this 768 dimensional matrix to map a value to each word.

3.2 Greedy Algorithms

While not a part of the machine learning curriculum, building an algorithm that uses the model that then greedily selects the best pairs based on similarity was the way to go for this assignment. The reality is that I tried my hand at an RNN model or using K-means clustering to try and make use of as many machine learning tools as I could, but with 16 data points per guess and the premise of an arbitrary semantic game, my goal was just to make the best possible guess by ‘testing’ the abilities of a sentence transformer. I did not want to further complicate our guesses.

Here is what the algorithm itself is doing:

```
# predict_groups() takes 16 words and their embeddings

#   1. Build a cosine similarity matrix across all remaining words
#   2. Try each word as a "seed" and greedily pull in its 3 most similar
neighbors
#   3. Score the candidate group by mean pairwise similarity (tighter =
higher score)
#   4. Commit the best-scoring group, remove those words, and repeat for
the next group
```

The cosine similarity score is what allows us to compare words to one another, we can loop through each word, find its score for the closest 3 other words, and based on how close those guesses were, make a confidence score for the given guess. It looks something like this in code:

```
sim_matrix = cosine_similarity(embeddings)

       # Try each remaining word as a seed and greedily pick its 3 most
similar neighbors
       for seed_idx in remaining_list:
           sims = [(j, sim_matrix[seed_idx][j])
                   for j in remaining_list if j != seed_idx]
           sims.sort(key=lambda x: -x[1])
           group = [seed_idx] + [j for j, _ in sims[:3]]
           # Score the group by mean pairwise similarity (higher = tighter
cluster)
           group_sims = [sim_matrix[i][j]
                         for i in group for j in group if i != j]
           score = np.mean(group_sims)
           if score > best_score:
               best_score = score
               best_group = group
```

3.3 Testing

What follows from this algorithm is to let it run independently on each puzzle, or more specifically on each individual group (four groups per puzzle) and see how it performs. To evaluate it, we make the following code:

```
def evaluate(data, word2idx, embedding_matrix):
    correct, total = 0, 0
    for puzzle in data:
        words = [m for ans in puzzle['answers'] for m in ans['members']]
        true_groups = [set(ans['members']) for ans in puzzle['answers']]
        embeddings = np.array([embedding_matrix[word2idx[w]] for w in
words])

        # Predict groups and check each against the true groupings
        predictions = predict_groups(words, embeddings)
        for pred in predictions:
            if set(pred['words']) in true_groups:
                correct += 1
            total += 1

    print(f"Groups correct: {correct}/{total} ({correct/total:.2%})")

evaluate(data, word2idx, embedding_matrix)
Groups correct: 586/4176 (14.03%)
```

And, we get a grand total of about 14% of puzzles are correct. This is not bad at all but there were some clear bugs in my code.

4. Adjusting for a higher success rate

4.1 One-Away Detection

The NYT Connections game notifies the player when a wrong guess contains exactly 3 of the 4 correct words. This is an important piece of information that could help my scores. If my algorithm makes a guess that is three out of four correct, I tell the algorithm to keep its guess and try removing its least correlated word.

4.2 Word Pool Restoration

A key bug that I found: when I made guesses that were either incorrect or dropped from the previous one-away detection logic, it would no longer let my model use it for its groupings. The fix was simple, when we drop words we need to restore them after we make a correct guess. This resulted in more fully correct puzzles.

```
def play_connections(puzzle, word2idx, embedding_matrix):
    words = [m for ans in puzzle['answers'] for m in ans['members']]
    true_groups = [set(ans['members']) for ans in puzzle['answers']]
    remaining_words = words.copy()
    dropped_words = []  # words temporarily removed

    correct = 0
    attempts = 0

    print(f"\nPuzzle #{puzzle['id']} ({puzzle['date']})")
    print(f"Words: {words}\n")

//The rest of the logic for playing the game below
```

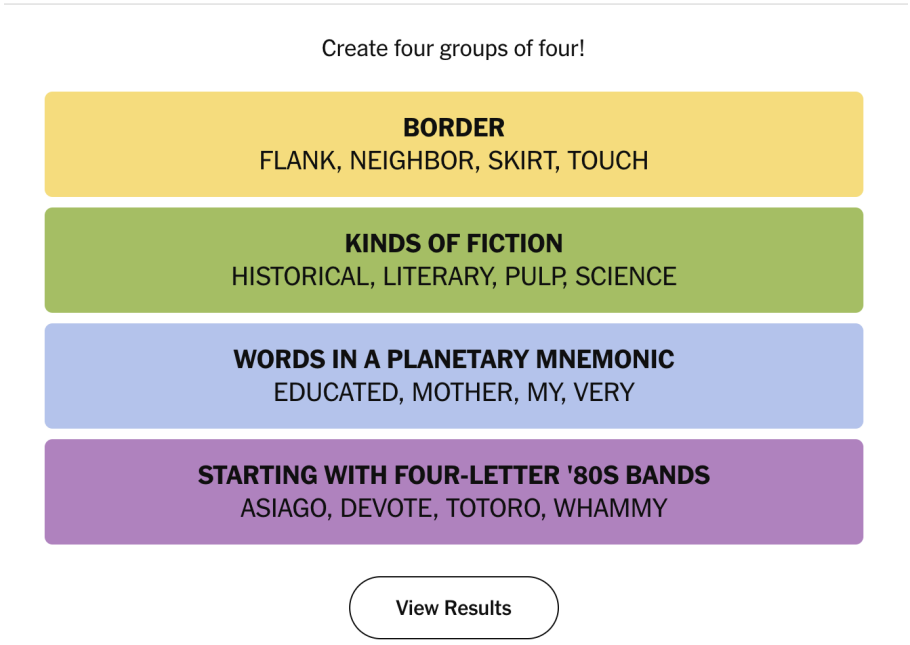
When I make an abstract function along the lines of “play game” with use of these two fixes, we see an instant jump to 23% success! This is huge!

5. Results

5.1 Performance Summary

Approach	Group Accuracy	Perfect Puzzles
Baseline Model	14.03%	—
Category based	23.41%	3.02%
Word Restoration	14.03%	10.23%
Best Version	23.41%	10.23%

6. Conclusion



The New York Times Connections is an extremely hard game, the screenshot above was one from Apr 23, 2026 and it's clear that no sentence transformer has the ability to figure out a pattern as obscure as this. Nonetheless, this project applied pretrained semantic embeddings and greedy algorithms reaching 23.41% group accuracy and 10.23% perfect puzzle rate without any task-specific training. Which is quite impressive. I am not finished with this project, several of my roommates have expressed that they wish to know just how well LLM's would do in comparison to a simple sentence transformer. I will continue to work on this and produce and present what I find in a unique way.